AWS CloudFormation

Real-World 3-Tier Infrastructure Project

Step-by-Step Practical Deployment Method

Infrastructure as Code (IaC) using AWS CloudFormation

VPC • Subnets • Internet Gateway • NAT Gateway • Security Groups • EC2 • Auto Scaling • ALB • RDS MySQL • Nested Stacks • GitHub • Jenkins

Prepared from the uploaded project guide.

1. Project Overview

The project creates a production-style multi-tier AWS environment using CloudFormation instead of creating resources manually in the AWS Console. The final architecture contains networking, application servers, load balancing and a private MySQL database.

```
INTERNET<br/>    |<br/>    v<br/>   ALB<br/>
    |<br/>    +---- EC2-A<br/>    
|<br/>    +---- EC2-B<br/>        
   |<br/>           v<br/>&nb
sp;      RDS MySQL<br/>       
;(Private)
```

Main infrastructure components:

• VPC with 2 Availability Zones

• 2 Public Subnets and 2 Private Application Subnets

• Internet Gateway and NAT Gateway(s)

• Route Tables and Security Groups

• Application Load Balancer and Target Group

• Launch Template and Auto Scaling Group

• EC2 application instances

• RDS MySQL in private subnets

• CloudFormation Parameters, Outputs, Conditions and Mappings

• Nested Stacks for modular templates

2. Project Folder Structure

Create the project with separate templates so each layer can be maintained and tested independently.

```
cloudformation-project/<br/>■■■ master.yaml<br/>■■■ 01-network.yaml<br/>■■■ 02-se
curity.yaml<br/>■■■ 03-database.yaml<br/>■■■ 04-compute.yaml<br/>■■■ 05-loadbalan
cer.yaml
```

Purpose:

File	Purpose
master.yaml	Main orchestration template
01-network.yaml	VPC and networking
02-security.yaml	Security Groups
03-database.yaml	RDS MySQL
04-compute.yaml	EC2 / Launch Template / Auto Scaling
05-loadbalancer.yaml	ALB and Target Group

3. Deployment Order

Follow this order. The network must exist before resources that depend on it.

```
1. Network<br/>2. Security<br/>3. Database<br/>4. Compute<br/>5. Load 
;Balancer<br/>6. Master Stack / final orchestration
```

4. Stage 1 – Create the VPC and Subnets

Create one VPC and divide it across two Availability Zones. Each AZ has one public subnet and one private application subnet.

```
VPC:             10.0.0.0/16<br/>Public Subnet A:   10.0.1.0/24<br/>Public Subnet B:   10.0.2.0/24<br/>Private Subnet A:  10.0.11.0/24<br/>Private Subnet B:  10.0.12.0/24
```

Steps:

1. Create the VPC with CIDR 10.0.0.0/16.
2. Select two Availability Zones in the chosen AWS Region.
3. Create Public Subnet A and Public Subnet B.
4. Create Private Subnet A and Private Subnet B.
5. Output the VPC ID and subnet IDs for other stacks.

5. Stage 2 – Create the Internet Gateway

Attach an Internet Gateway to the VPC and use it for public subnet Internet connectivity.

1. Create the Internet Gateway.
2. Attach it to the VPC.
3. Create/associate the public route table with the public subnets.
4. Add the default route 0.0.0.0/0 to the Internet Gateway.

```
Public Route<br/>0.0.0.0/0  --->  Internet Gateway
```

6. Stage 3 – Create NAT Gateway

Private EC2 instances should not be directly reachable from the Internet, but they may need outbound Internet access for package installation, updates, downloads and API access.

1. Allocate an Elastic IP for the NAT Gateway.
2. Create the NAT Gateway in a public subnet.
3. Create a private route table.
4. Add 0.0.0.0/0 through the NAT Gateway.
5. Associate the private subnets with the private route table.

```
Private EC2<br/>    |<br/>    v<br/>NAT Gateway<br/>    |<br/>    v<br/>Internet Gateway<br/>    |<br/>    v<br/>Internet
```

7. Stage 4 – Create Security Groups

Use separate Security Groups for the ALB, application servers and database.

ALB Security Group:

Allow HTTP 80 and HTTPS 443 from the Internet when required.

Application Security Group:

Allow application traffic only from the ALB Security Group (for example TCP 80).

Database Security Group:

Allow MySQL TCP 3306 only from the Application Security Group. Do not expose port 3306 directly to the Internet.

```
Internet<br/>   |<br/>   v<br/>ALB-SG<br/>   |<br/> TCP 80<br/>   v<br/>APP-SG<br/>   |<br/> TCP 3306<br/>   v<br/>DB-SG
```

8. Stage 5 – Create EC2 Launch Template

Use a Launch Template instead of manually creating individual EC2 instances.

Define the AMI, instance type, security group, IAM instance profile, EBS volume, User Data and tags.

Steps:

1. Select the required AMI for the application.
2. Select an instance type such as t3.micro for the example project.
3. Attach the Application Security Group.
4. Configure IAM Instance Profile if required.
5. Add User Data to install/start the application.
6. Add useful Name/Environment tags.

9. Stage 6 – Create Auto Scaling Group

The Auto Scaling Group maintains the required number of application instances and can replace failed instances.

```
Desired Capacity = 2<br/>Minimum Capacity = 2<br/>Maximum Capacity = 4
```

Steps:

1. Create the Auto Scaling Group using the Launch Template.
2. Select both private application subnets.
3. Set desired capacity to 2.
4. Set minimum capacity to 2 and maximum to 4.
5. Configure health checks as required.

10. Stage 7 – Create Application Load Balancer

The ALB receives user traffic and distributes it to the EC2 instances through a Target Group.

Steps:

1. Create the ALB in the public subnets.
2. Attach the ALB Security Group.
3. Create a Target Group for the application instances.
4. Configure the listener, commonly HTTP port 80 for the basic project.
5. Register the Auto Scaling Group/instances with the Target Group.
6. Configure health checks.

```
User<br/>  |<br/>  v<br/> ALB<br/>  |<br/>  v<br/>Target Group<br/>  |<br/>  +---- EC2-A<br/>  |<br/>  +---- EC2-B
```

11. Stage 8 – Create RDS MySQL

Place the database in private subnets and use a DB Subnet Group containing the private subnets.

Steps:

1. Create the DB Subnet Group using Private Subnet A and Private Subnet B.
2. Create the RDS MySQL instance.
3. Attach the Database Security Group.
4. Allow TCP 3306 only from the Application Security Group.
5. Configure backups and Multi-AZ according to the environment. Production environments should use appropriate backup and availability settings.

12. Add CloudFormation Parameters

Parameters make templates reusable without editing the YAML each time.

```
Parameters:<br/>  InstanceType:<br/>    Type: String<br/>    Default: t3.micro<br/><br/>  EnvironmentName:<br/>    Type: String<br/>    Default: dev
```

When creating the stack, parameter values can be changed as needed.

13. Add CloudFormation Resources

Resources define the AWS infrastructure CloudFormation should create.

```
Resources:<br/>  MyVPC:<br/>    Type: AWS::EC2::VPC<br/>    Properties:<br/>      CidrBlock: 10.0.0.0/16
```

14. Add CloudFormation Outputs

Outputs expose useful values after deployment. Recommended outputs include VPC ID, subnet IDs, security group IDs, Load Balancer DNS and RDS endpoint.

```
Outputs:<br/>  VPCId:<br/>    Description: VPC ID<br/>    Value: !Ref MyVPC
```

15. Use Important CloudFormation Functions

!Ref – returns a resource reference value.

!GetAtt – gets an attribute such as an ALB DNS name.

!Sub – builds strings using variables.

!FindInMap – retrieves a value from a Mapping.

```
VpcId: !Ref MyVPC<br/><br/>Value: !GetAtt MyLoadBalancer.DNSName<br/><br/>Value: !Sub '${MyLoadBalancer.DNSName}:80'<br/><br/>ImageId: !FindInMap<br/>  - RegionMap<br/>  - !Ref AWS::Region<br/>  - AMI
```

16. Add Mappings

Mappings can store region-specific or environment-specific values.

```
Mappings:<br/>  EnvironmentMap:<br/>    dev:<br/>      InstanceType: t3.micro<br/>    test:<br/>      InstanceType: t3.small<br/>    production:<br/>      InstanceType: t3.medium
```

Use !FindInMap to retrieve the value for the selected environment.

17. Add Conditions

Conditions allow resources to be created only when a condition is true.

```
Conditions:<br/>  IsProduction:<br/>    !Equals<br/>      - !Ref EnvironmentName<br/>      - production
```

Then add Condition: IsProduction to a resource that should exist only in production.

18. Create Nested Stacks

For a larger project, use a master template to orchestrate the individual templates.

```
master.yaml<br/>   |<br/>   +-- 01-network.yaml<br/>   +-- 02-security.yaml<br/>   +-- 03-database.yaml<br/>   +-- 04-compute.yaml<br/>   +-- 05-loadbalancer.yaml
```

This modular structure makes the project easier to maintain, test, update, troubleshoot and reuse.

19. Create the CloudFormation Stack in AWS Console

1. Open AWS Console.

2. Open CloudFormation.

3. Select Stacks → Create stack → With new resources.

4. Choose Template is ready.

5. Choose Upload a template file.

6. Upload the YAML template.

7. Enter a stack name, for example production-infrastructure.

8. Click Next → Next → Next → Submit.

20. Monitor Stack Status

Important statuses include CREATE_IN_PROGRESS, CREATE_COMPLETE, CREATE_FAILED, UPDATE_COMPLETE, UPDATE_FAILED, ROLLBACK_COMPLETE and DELETE_COMPLETE.

During troubleshooting, always open the CloudFormation Events tab and inspect the Status reason.

21. Troubleshoot CREATE_FAILED

Open CloudFormation → Stacks → Your Stack → Events. Find the CREATE_FAILED resource and read its Status reason.

Common checks:

• Key Pair error: verify that the key pair exists in the selected AWS Region and that KeyName is correct.

• Default VPC error: create and reference the VPC explicitly in CloudFormation.

• Availability Zone error: make sure the selected AZs exist in the chosen Region.

• Security Group error: check VPC ID, CIDR, port and Security Group references.

22. Use Change Sets Before Production Updates

```
Old Template<br/>     |<br/>     v<br/>New&n
bsp;Template<br/>     |<br/>     v<br/>Change&nbs
p;Set<br/>     |<br/>     v<br/>Review Chang
es<br/>     |<br/>     v<br/>Execute
```

A Change Set lets you review what CloudFormation plans to modify before executing the update.

23. Understand Rollback

If a deployment fails, CloudFormation can roll back the failed operation and may remove resources created during that operation.

```
CREATE<br/>  |<br/>  +-- VPC       SUCC
ESS<br/>  +-- Subnet    SUCCESS<br/>  +-- Securit
y  FAILED<br/>  |<br/>  v<br/>ROLLBACK
```

24. Store the Project in GitHub

```
GitHub<br/>  |<br/>  +-- master.yaml<br/>  +-- 01-network.y
aml<br/>  +-- 02-security.yaml<br/>  +-- 03-database.yaml<br/> &
nbsp;+-- 04-compute.yaml<br/>  +-- 05-loadbalancer.yaml
```

Benefits: version control, change history, collaboration, code review and CI/CD integration.

25. Connect CloudFormation with Jenkins

```
Developer<br/>   |<br/>   v<br/>GitHub<br/>   |<br/>&n
bsp;  v<br/>Jenkins<br/>   |<br/>   v<br/>Validate Clo
udFormation<br/>   |<br/>   v<br/>Create Change Set<br/>&nb
sp;  |<br/>   v<br/>Review<br/>   |<br/>   v
<br/>Deploy<br/>   |<br/>   v<br/>AWS
```

Possible validation tools include cfn-lint, CloudFormation validate-template and Change Sets.

26. Final Verification Checklist

■ VPC created with correct CIDR.

■ Two Availability Zones configured.

■ Public and private subnets created.

■ Internet Gateway attached and public routing configured.

■ NAT Gateway and private routing configured.

■ ALB, application and database Security Groups separated.

■ Launch Template created.

■ Auto Scaling Group running the desired EC2 capacity.

■ ALB Target Group and health checks working.

■ RDS MySQL is private and reachable only from the application layer.

■ Parameters, Outputs, Conditions and Mappings added where needed.

■ Nested/master stack structure works.

■ CloudFormation stack reaches CREATE_COMPLETE.

■ CloudFormation Events show no unexpected failures.

■ Templates stored in GitHub.

■ Jenkins validation/deployment flow is ready if CI/CD is required.

27. Final Project Goal

The final goal is to run master.yaml and have CloudFormation create the complete environment: VPC, two Availability Zones, public/private subnets, Internet Gateway, NAT Gateway, Security Groups, ALB, Target Group, Launch Template, Auto Scaling Group with EC2 instances, and private RDS MySQL.

```
master.yaml<br/>   |<br/>   +-- VPC<br/>   +-- 2 Availability Zones<br/>   +-- Public Subnets<br/>   +-- Private Subnets<br/>   +-- Internet Gateway<br/>   +-- NAT Gateway<br/>   +-- Security Groups<br/>   +-- ALB<br/>   +-- Target Group<br/>   +-- Launch Template<br/>   +-- Auto Scaling Group<br/>   |     +-- EC2<br/>   |     +-- EC2<br/>   +-- RDS MySQL
```

Project progression from the source guide:

Phase 1: VPC, Subnets, Internet Gateway, Route Tables

Phase 2: NAT Gateway, Private Routing

Phase 3: Security Groups

Phase 4: EC2, Launch Template, Auto Scaling Group

Phase 5: Application Load Balancer, Target Group, Health Checks

Phase 6: RDS MySQL, DB Subnet Group, Database Security

Phase 7: Nested Stacks, Parameters, Mappings, Conditions, Outputs

Phase 8: GitHub, Jenkins, CloudFormation CI/CD

Source basis

This PDF is organized from the uploaded project guide, which describes the 3-tier CloudFormation architecture, deployment stages, stack operations, troubleshooting, Change Sets, Git and Jenkins integration. ■filecite■turn0file0■L1-L7■